10 Day Churn Prediction

Authors:

Licheng Xiong

Ziqi Meng



1. Problem statement

We predict whether a user will churn **within 10 days after an observation cutoff**. Churn is defined as the user visiting the page **"Cancellation Confirmation"** during the 10-day prediction window.

Binary target for a user u at cutoff c:

$$y(u,c)=\mathbf{1}\{\exists \text{ event of u with page = "Cancellation Confirmation" in (c, c + 10 days)}\}$$

The test set contains users observed up to **2018-11-20**, and we predict churn in the subsequent 10 days.

2. Data overview

The dataset contains **event-level user interaction logs**. Each row is one user event (page view / song play / etc.) with timestamp and contextual attributes.

- **Dataset profiling:** We observe a large number of missing cells in the raw event logs. However, a closer inspection reveals that these missing values are structural rather than due to data quality issues.

Number of variables	Number of observations	Missing cells	Duplicate rows
19	17499636	9624609	0

- **Missing values are "structural missing"**:

A large fraction of missing values is structural: when an event is not a song-play interaction, the attributes `length`, `song`, and `artist` are not applicable and are missing by design. The missing count matches exactly, (17,499,636 - 14,291,433) × 3 = 9,624,609, confirming there are no random data-quality missing values that require imputation.

- **Variable types:**

We use low-to-moderate cardinality categorical variables for feature derivation, including: `status, gender, level, auth, page, method`.

Additionally, the text field `location` follows a consistent pattern; the last two characters correspond to U.S. state abbreviations. We extract **state** as a categorical feature.

3. Training set construction (sliding-window)

3.1 Sliding-window reasons

Because the test set contains users observed up to a fixed date with no observed cancellation at prediction time, the task is to predict whether a user will churn within the following 10 days.

To faithfully reproduce this information structure during training, we construct training samples that mimic the test setting. In particular, at each prediction point, users must be "alive" (i.e., not yet cancelled), and labels are defined exclusively based on future events.

Based on this analysis, we considered three possible strategies to construct training labels:

(1) Direct labeling.

- Users are labeled as churned if they ever visit the Cancellation Confirmation page in the training logs.

- However, this approach **ignores the 10-day prediction horizon** and introduces severe label leakage.

(2) Single fixed cutoff.

- Features are computed using events before the **fixed** cutoff.
- Labels are defined based on the subsequent 10-day window.
- While this respects the prediction horizon, it **discards all churn events occurring before the cutoff**, leading to a substantial loss of information.

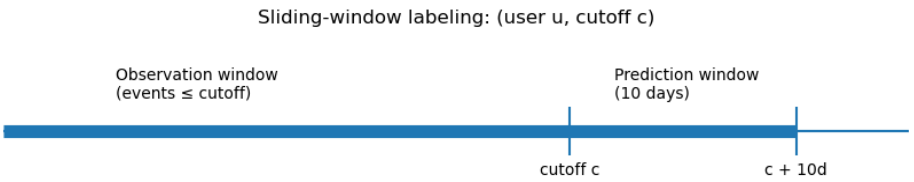
(3) Sliding cutoff.

- Features are computed using events before the **sliding** cutoff, generating multiple samples per user.
- Labels are defined based on the subsequent 10-day window.
- This approach preserves the same information structure as the test set, fully exploits available churn events, and avoids using future information.

3.2 Sliding-window constructions

More specifically, we construct training examples with a sliding cutoff date approach as:

- For each cutoff date c, we use only events with ts <= c to build features.
- The label is computed from events in (c, c + 10 days].
- We only create samples for users who are still “active” by cutoff (to avoid labeling users who already churned before the cutoff).



For each cutoff date c, define horizon H=10 days. The label is: y(u,c)=1{∃ event of user u with page = "Cancellation Confirmation" in (c,c+H]}

We slide cutoff dates daily over a feasible range from **2018-10-15** to **2018-11-09** because:

- c should start after enough history exists
- c should end at cmax = last_available_train_date−(H+1)
  so every cutoff has a complete future 10-day window inside the training logs.

To ensure the sample is eligible in the sliding-window, we impose the following constraints when we create a sample (u,c):

- the user has observed history up to c
- the user has **not churned at or before c**.

4. Feature Engineering

Section / Category	Feature Variables & Calculation Logic
4.1 Static Attributes	• User Profile: gender, state
4.2 Lifecycle	• Tenure: days_since_registration

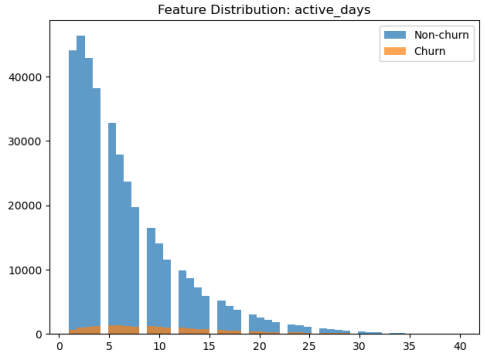
Section / Category	Feature Variables & Calculation Logic
4.3 Recency & Activity	• General: recency_hours (time since last act), n_events (total), active_days, total_listen_time • Rolling Windows (1d/3d/7d): – events_last_[1/3/7]d – songs_last_[1/3/7]d
4.4 Session Behavior	• Count: session_count • Duration Stats: [mean, max, min, std] of session duration • Intensity Stats: [mean, max, min, std] of events-per-session
4.5 Page Interaction	• Metrics: cnt_page_*, ratio_page_* • Logic: Computed on unified page vocabulary (train + test set); strictly uses events up to cutoff to prevent leakage.
4.6 Subscription	• Status: level_at_cutoff (latest known level) • History: n_level_change (count of switches), ever_paid (binary flag)
4.7 Technical Status	• HTTP Signals: Counts & Fractions of specific status codes – e.g., frac_status_404, cnt_status_200, cnt_status_307

Several features are designed to capture user engagement and experience quality.

- User lifetime reflects tenure on the platform, where longer lifetimes typically correspond to more established users who are less likely to churn.
- Session-level features describe the depth of user engagement; longer or more intensive sessions often indicate stronger immersion and higher retention.
- Status signals provide proxies for user experience, where frequent error responses (e.g., HTTP 404) may lead to frustration and increase the likelihood of churn.

5. Exploratory plots

Distribution comparison for key features (target=0 vs target=1)



Many count- and duration-based features exhibit heavy-tailed, right-skewed distributions. To mitigate the influence of extreme values and stabilize feature scales, we apply log(1+x) transformations to these variables.

6. Handling imbalance

- Based on descriptive statistics of the training data, we observe that before applying the sliding-window construction, approximately **22%** of users eventually churn. However, after generating training samples using sliding cutoffs, the positive rate drops to around **5%**, as each churn event is contrasted with many non-churn cutoffs.

- This results in a highly imbalanced classification problem. To mitigate this issue, we apply oversampling of the minority class when training linear models, in particular Logistic Regression, to improve stability and sensitivity to churn events.

7. Models

7.1 Start with single-model

- We first evaluated several single-model approaches with default parameters, including Logistic Regression, tree-based models (LightGBM, Random Forest, Extra Trees), SVM, and KNN.

- Among them, KNN consistently underperformed. This is expected given the high-dimensional and heterogeneous feature space (counts, ratios, one-hot encoded variables), where distance-based methods suffer from the curse of dimensionality. KNN was therefore excluded from further consideration.

- Logistic Regression, SVM and tree-based models showed the most stable and competitive performance as single models. We then explored ensemble methods by combining different model families.

7.2 Construct ensembles

- We found that ensembles combining Logistic Regression with tree-based models perform particularly well, as the two model types are complementary: Logistic Regression provides stable, well-calibrated global rankings, while tree-based models effectively capture non-linear patterns and are robust to feature correlation induced by the sliding-window construction.

- In contrast, combining SVM with other models did not yield consistent improvements, likely due to its sensitivity to hyperparameters and reduced robustness at large sample sizes. As a result, SVM was excluded from the final ensemble.

7.3 Tune the final model

- Based on these observations, we adopt a soft-voting ensemble combining Logistic Regression, LightGBM, and Random Forest as our final model.

- To obtain optimal performance each base model is first tuned individually using grid search on the training data, and the resulting optimized models are then combined through soft voting.

8. Final training and prediction

- For the final submission, we convert predicted probabilities into binary labels using a Top-50% rule, i.e., we label as churn the 50% users with the highest predicted churn probabilities, to match the expected class balance in the test set.
- Our final submission achieved **0.692** on the **Public Leaderboard** (ranked **3rd**) and **0.642** on the **Private Leaderboard** (ranked **5th**).